

Lab 4: Create an Amazon S3 Bucket

AWS Technical Essentials

Duration: ~20 minutes

🏷️ Resource Names for This Lab

Replace <ID> with your assigned student ID (for example `user7`).

Resource	Name
S3 bucket	`employee-directory-app-bucket-<ID>-<RANDOM>`

> ⚠️ **Two layers of uniqueness:** your <ID> keeps you apart from your classmates, and <RANDOM> (any 4-6 digits or letters) keeps the name unique across every AWS account on earth.

🎯 Lab Overview

📋 Objectives

- Create an S3 bucket with versioning
- Upload objects and test access
- Generate a presigned URL
- Enable static website hosting
- Add a public bucket policy

📝 Lab Details

- **Duration:** ~20 minutes
- **Region:** `us-east-1` (N. Virginia) — same for every lab

> ⚠️ Bucket names must be **globally unique** across all AWS accounts.

📦 Create a Bucket with Versioning

1. Navigate to **S3** in the console
2. Click **Create bucket**
3. Bucket name: `employee-directory-app-bucket-<ID>-<RANDOM>`
4. Region: **US East (N. Virginia) `us-east-1`**
5. Block Public Access: **Keep all checked** (for now)
6. Bucket Versioning: **Enable**
7. Encryption: **SSE-S3** (default)
8. Click **Create bucket**

> ✅ **Expected:** Green success banner confirming bucket creation.

📤 Upload Objects

1. Click on your bucket name to open it
2. Click **Upload**
3. Click **Add files** and select 2-3 small files from your computer
4. Click **Upload**, then **Close**

> ✅ **Expected:** Files appear in the objects list with their size and storage class (Standard).

🔗 Test Access: Object URL vs Presigned URL

1. Click on any uploaded object name
2. Copy the **Object URL** and open it in a new tab

> ❌ **Expected: Access Denied** — Block Public Access is enabled.

3. Back in the console, select the object checkbox
4. Click **Actions** → **Share with a presigned URL**
5. Set duration to **5 minutes**, click **Create presigned URL**
6. Open the presigned URL in a new tab

> ✅ **Expected:** The object loads. Presigned URLs include temporary credentials in the query string.

🌐 Enable Static Website Hosting

1. Go to your bucket's **Properties** tab
2. Scroll to **Static website hosting**, click **Edit**
3. Select **Enable**
4. Index document: `index.html`
5. Click **Save changes**
6. Note the **Bucket website endpoint** URL

Create a file named `index.html` on your computer with:

```
<!DOCTYPE html>
<html>
<head><title>Employee Directory</title></head>
<body>
  <h1>Employee Directory Application</h1>
  <p>Welcome to the Employee Directory static site
     hosted on S3!</p>
</body>
</html>
```

7. Upload `index.html` to your bucket

🔓 Add Bucket Policy for Public Access

1. Go to **Permissions** tab
2. Under **Block public access**, click **Edit**, **uncheck all**, Save, type `confirm`
3. Under **Bucket policy**, click **Edit** and paste:

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "PublicReadGetObject",
            "Effect": "Allow",
            "Principal": "*",
            "Action": "s3:GetObject",
            "Resource": "arn:aws:s3:::BUCKET_NAME/*"
        }
    ]
}
```

4. Replace BUCKET_NAME with your actual bucket name
5. Click **Save changes**

> ⚠️ **Production note:** Only disable Block Public Access for intentionally public content. Use CloudFront + OAC in production.

🌐 Test the Website Endpoint

1. Go to **Properties** tab
2. Scroll to **Static website hosting**
3. Click the **Bucket website endpoint** URL

> ✅ **Expected:** Your browser displays the "Employee Directory Application" page. You are hosting a static website on S3.

🗑️ Clean Up

Delete the S3 bucket and its contents:

1. Go to the S3 bucket list
2. Select your bucket, click **Empty**, type *permanently delete*, confirm
3. Select your bucket again, click **Delete**, type the bucket name, confirm

> ⚠️ **Note:** You must empty a bucket before you can delete it. With versioning enabled, "Empty" removes all object versions and delete markers.

🎉 Lab 4 Complete!

Amazon S3 Bucket with Static Website

🎯 Key Takeaways

🛠️ What You Did

- Created an S3 bucket with versioning
- Uploaded objects and tested access
- Generated a presigned URL
- Hosted a static website on S3

📚 Key Concepts

- **Block Public Access** prevents accidental exposure
- **Presigned URLs** provide temporary access
- **Bucket policies** control resource-level access
- **Static hosting** serves web content from S3



Questions?

Ask your instructor before proceeding to the next module